

Tutorial 5: Database and SQL Interaction

1. Introduction

This tutorial introduces working with relational databases using Python. You will practice storing, querying, and modifying structured datasets using SQLite, SQL, and SQLAlchemy.

You will learn to:

- Understand structured datasets and relational tables
- Create and query SQLite databases
- Perform insert, update, and delete operations
- Use SQLAlchemy ORM

2. Structured Data and Relational Databases

2.1 Structured Data

Structured data is organized in rows and columns and follows a fixed schema.

Example table:

id	name	age
1	Ram	22
2	Sita	24

2.2 Key Concepts

- Table – Collection of records
- Row – Single record
- Column – Attribute
- Primary Key – Unique identifier
- Foreign Key – Link between tables
- Schema – Structure definition

3. Working with SQLite in Python

3.1 Connecting to SQLite

```
import sqlite3

conn = sqlite3.connect("students.db")
cursor = conn.cursor()
```

3.2 Creating a Table

```
cursor.execute("""
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER,
    grade TEXT
)
""")
conn.commit()
```

3.3 Inserting Data

```
cursor.execute(
    "INSERT INTO students (name, age, grade) VALUES"
    "(?, ?, ?)",
    ("Asha", 21, "A")
)
conn.commit()
```

3.4 Querying Data

```
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()

for row in rows:
    print(row)
```

3.5 Updating Data

```
cursor.execute(  
    "UPDATE students SET grade=? WHERE name=?",  
    ("A+", "Asha")  
)  
conn.commit()
```

3.6 Deleting Data

```
cursor.execute(  
    "DELETE FROM students WHERE name=?",  
    ("Asha",)  
)  
conn.commit()
```

3.7 Closing Connection

```
conn.close()
```

4. Introduction to SQLAlchemy

SQLAlchemy allows interaction with databases using Python objects rather than raw SQL.

Installation:

```
pip install sqlalchemy
```

4.1 Creating Engine

```
from sqlalchemy import create_engine
```

```
engine = create_engine("sqlite:///school.db")
```

4.2 Defining Tables with ORM

```
from sqlalchemy.orm import declarative_base  
from sqlalchemy import Column, Integer, String
```

```
Base = declarative_base()
```

```
class Student(Base):  
    __tablename__ = "students"  
    id = Column(Integer, primary_key=True)
```

```
name = Column(String)
age = Column(Integer)
grade = Column(String)
```

```
Base.metadata.create_all(engine)
```

4.3 Adding Records

```
from sqlalchemy.orm import sessionmaker
```

```
Session = sessionmaker(bind=engine)
session = Session()
```

```
new_student = Student(name="Hari", age=23, grade="B")
session.add(new_student)
session.commit()
```

4.4 Querying Records

```
students = session.query(Student).all()
```

```
for s in students:
    print(s.name, s.age, s.grade)
```

4.5 Updating Records

```
student =
session.query(Student).filter_by(name="Hari").first()
student.grade = "A"
session.commit()
```

4.6 Deleting Records

```
session.delete(student)
session.commit()
```

Exercises

1. Create a SQLite database for a library system.
2. Insert at least five book records.
3. Write queries to retrieve books by author.
4. Update book availability status.
5. Delete a record.
6. Recreate the same schema using SQLAlchemy ORM.
7. Insert and query records using ORM.
8. Compare raw SQL and ORM approaches.